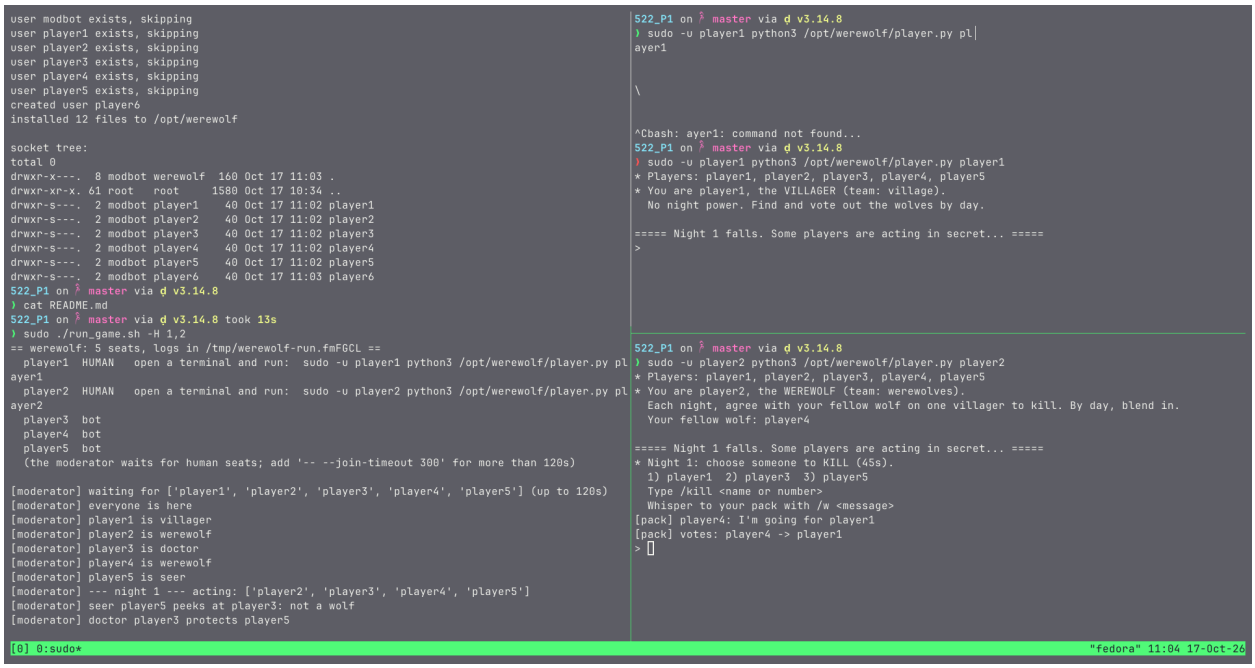
Cade McNally

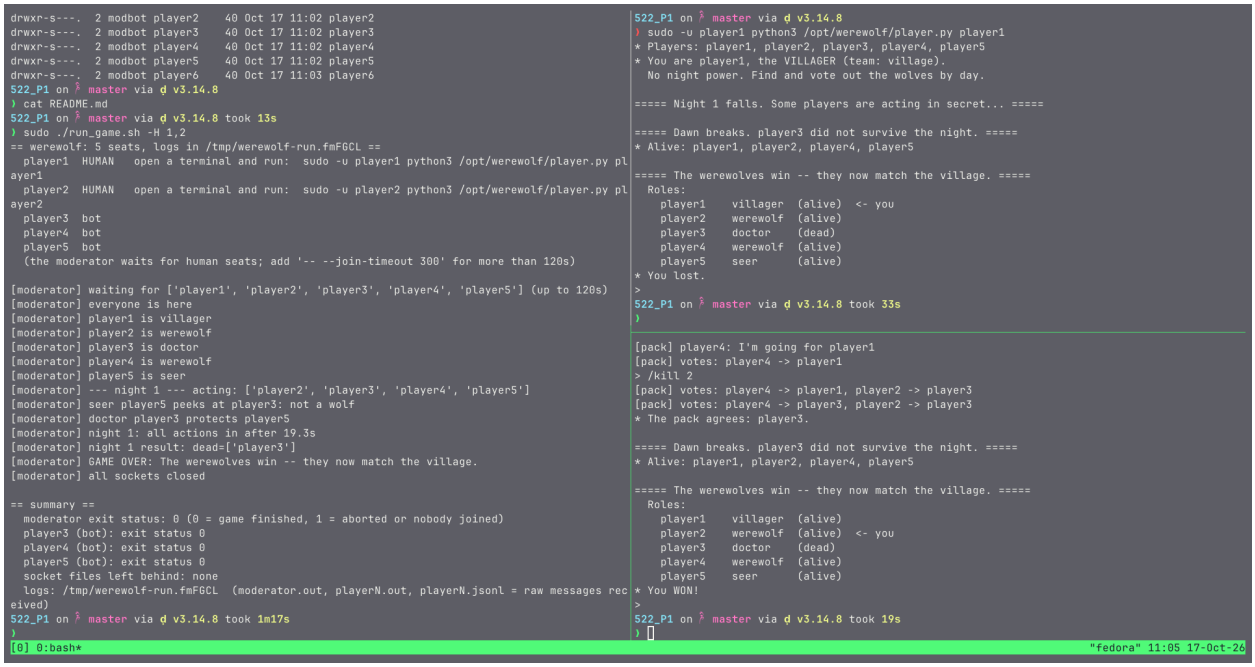
My werewolves project:

1. This game is simply a werewolves style inspired game played completely in the terminal. It can be played by bots, humans and bots, or just humans. There are villagers, werewolves, a doctor, and a seer. Each player (bot or human) has an equal chance at being assigned each. The moderator can see the results of this and actions taken by each player. There is a night phase for werewolf killing and doctor protecting, a phase for the villagers to discuss and choose a player to eliminate, and time for the seer to look for a certain players role. The communications library API is commslib.py, and it is the only file in the project that touches sockets.
2. 

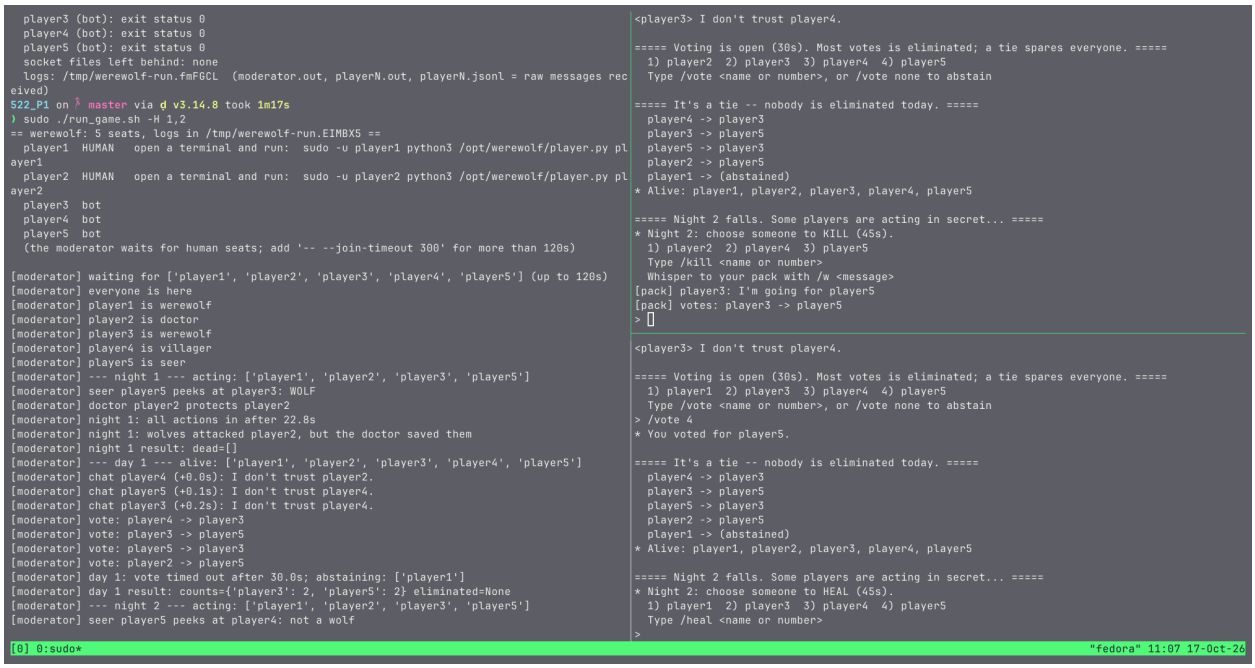
This picture shows the user/player setup and permissions testing before playing the game.



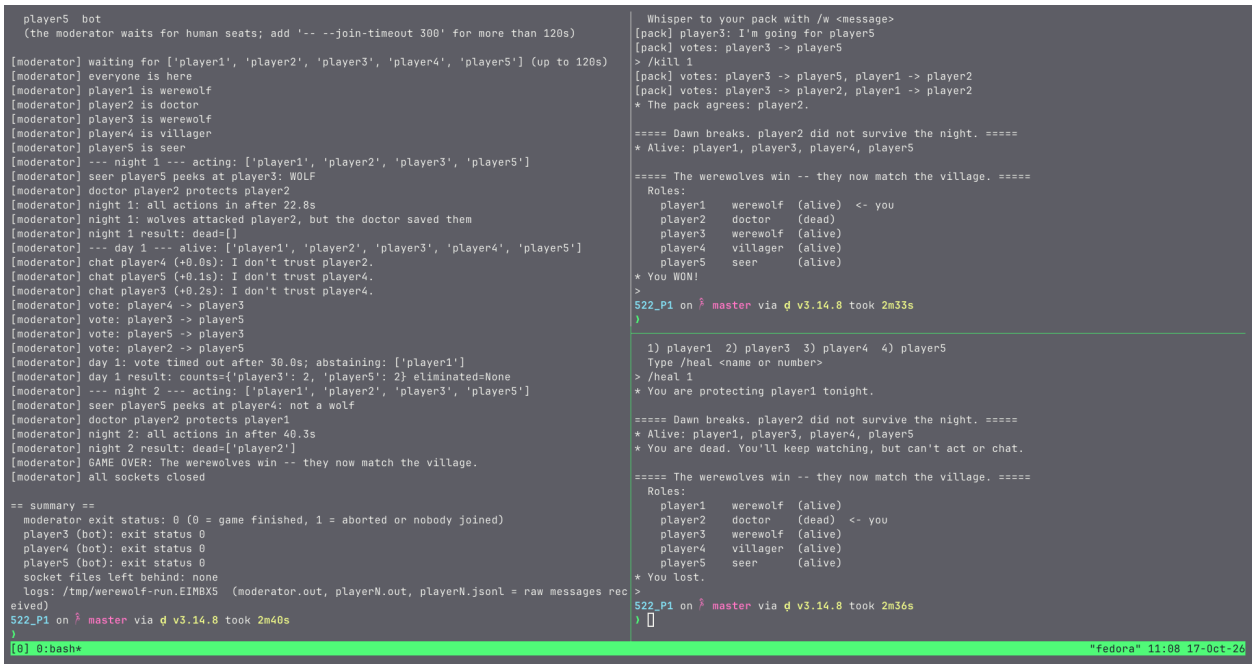
This picture shows the game starting, and the views of the moderator along with both human players.



This screenshot shows an early game win by the werewolves after voting a player to be killed.



This screenshot shows the doctor protecting a player that would otherwise have been killed by the werewolves.



This screenshot shows the wolves killing player 2 and the doctor choosing the wrong player to protect, resulting in a victory for the werewolves again.

3. My communications primitive is stored in the file commslib.py. The primitive that I chose was a UNIX domain stream socket approach. This approach allowed my game to use a TCP-like network stack without the connection to the network that TCP requires. It achieves this behavior due to the socket APU exchanging bytes via a kernel object. Each player is given a listening socket, with the moderator able to read the connecting

players/processes UID to confirm who is actually connecting. Each message is framed as a 4 byte big endian, followed by a UTF-8 JSON body, and the receiver uses ‘readexactly’ to get exactly one message at a time. In respect to multicasting, UNIX sockets do not have a built-in multicast. To handle this, the solution is to build per player connections, using ‘gather’ to send to the receivers at the same time. Due to this simplified approach/solution, this primitive was kept over starting over with other options such as localhost TCP or shared memory.

4. No feedback yet
5. Choosing the primitive method that I did was my idea in itself. I did not understand it at first, and the research I did left some doubts before implementation. I used AI to write the commslib.py file, reviewing what it did and searching for things I did not understand. The multicast solution that I discussed above was one such doubt and gap in understanding that I had prior to reviewing the code. The solution implemented worked very well and helped me understand the simplicity of this primitive. The primitive also supports mixed delivery; multicasting, 1:1, and broadcast. Prior to starting development, I was not sure on the ways this could be implemented. As stated above, the per player connections built for each receiver supports this behavior. Overall, I am glad to have chosen this primitive and will do more of my own research into it to continue the expansion of this game into future projects.

Feedback to others:

I was able to give feedback to a couple of classmates. I wanted to test the games out on a VM, but by my own fault I waited too long to be able to do this in a timely manner. Instead, I looked at the code provided and files including their communications primitive. From this I was able to formulate questions and general comments about functions implemented, approaches taken, and possible functionality (or lack thereof). For one of the classmates I responded to, a primitive I was not overly familiar with was chosen. For this case, I still reviewed the code and gave my thoughts and feedback to the best of my ability. In the future, I plan to be better in this area and give more relevant feedback with results from playing the game myself.